

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks:25

Annexure A

Experiments

Code of Conduct:

I am M. Vidhya (192412151) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Short Answer Test

1. Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

Sol)

1. Aim

To implement a **Naïve Bayes classifier** on a real-world dataset, compare different feature sets, calculate accuracy, precision and recall, and study the effect of feature independence and Laplace smoothing.

2. Theory

Naïve Bayes is a supervised classification algorithm based on **Bayes' theorem**.

For multiple features:

It assumes that the features are **conditionally independent** given the class.

3. Laplace Smoothing

To avoid zero probability for unseen values:

It assigns a small non-zero probability to unseen feature values.

4. Dataset

Iris dataset – 150 samples, 4 features and 3 classes.

Features: Sepal Length, Sepal Width, Petal Length, Petal Width.

5. Feature Sets

Set	Features
Set 1	Sepal Length, Sepal Width
Set 2	Petal Length, Petal Width
Set 3	All 4 features

6. Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, precision_score, recall_score

iris = load_iris()
X, y = iris.data, iris.target

sets = {
    "Sepal": [0, 1],
    "Petal": [2, 3],
    "All": [0, 1, 2, 3]
}

for name, f in sets.items():
    X1 = X[:, f]

    X_train, X_test, y_train, y_test = train_test_split(
        X1, y, test_size=0.2, random_state=42, stratify=y)

    model = GaussianNB()
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    print(name)
    print("Accuracy :", round(accuracy_score(y_test, y_pred), 4))
    print("Precision:", round(precision_score(y_test, y_pred, average='weighted'), 4))
    print("Recall  :", round(recall_score(y_test, y_pred, average='weighted'), 4))
```

Sample Output:

Sepal

Accuracy : 0.7

Precision: 0.7003

Recall : 0.7

Petal

Accuracy : 0.9667

Precision: 0.9697

Recall : 0.9667

All

Accuracy : 0.9667

Precision: 0.9697

Recall : 0.9667

8. Performance Metrics

Accuracy:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision:

$$Precision = \frac{TP}{TP + FP}$$

Recall:

$$Recall = \frac{TP}{TP + FN}$$

9. Analysis

- Petal features give better classification performance.
- Naïve Bayes assumes features are independent.
- Correlated features can affect probability estimation.
- Laplace smoothing prevents zero probabilities.
- Proper feature selection improves classification performance.

10. Result

The Naïve Bayes classifier was successfully implemented. **Petal features achieved better performance**, showing that suitable feature selection improves classification accuracy. Laplace smoothing helps avoid zero-probability problems and makes probability estimation more reliable.

2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

Sol)

1. Aim

To estimate the **mean and variance** of a probability distribution using **Maximum Likelihood Estimation (MLE)** and compare it with **Bayesian estimation** for different sample sizes.

2. Theory

Maximum Likelihood Estimation (MLE) estimates the parameters that make the observed data most likely.

For a normal distribution:

Bayesian estimation combines the observed data with prior knowledge.

Where:

- = Posterior probability
- = Likelihood
- = Prior probability

3. Dataset

Consider the following observations:

10, 12, 11, 13, 12, 14, 15, 11, 13, 12

We estimate the **mean and variance** of these observations.

4. Procedure

1. Load the dataset.
2. Calculate the mean and variance using MLE.

3. Assume a prior mean for Bayesian estimation.
4. Combine the prior information with the observed data.
5. Calculate the Bayesian mean.
6. Compare MLE and Bayesian estimates.
7. Repeat with different sample sizes.
8. Analyze the effect of the prior.

5. Python Program

```
import numpy as np
```

```
data = np.array([10, 12, 11, 13, 12, 14, 15, 11, 13, 12])
```

```
# MLE
```

```
mle_mean = np.mean(data)
```

```
mle_variance = np.var(data)
```

```
# Bayesian estimation
```

```
prior_mean = 12
```

```
prior_weight = 2
```

```
n = len(data)
```

```
bayesian_mean = (prior_weight * prior_mean + n * mle_mean) /\n                (prior_weight + n)
```

```
print("MLE Mean    :", round(mle_mean, 2))
```

```
print("MLE Variance :", round(mle_variance, 2))
```

```
print("Bayesian Mean :", round(bayesian_mean, 2))
```

Output:

```
MLE Mean    : 12.3
```

```
MLE Variance : 2.01
```

```
Bayesian Mean : 12.25
```

8. Effect of Sample Size

Sample Size	Effect of Prior
Small	Prior has more influence
Large	Data has more influence
Very Large	Bayesian estimate approaches MLE

9. Analysis

- MLE uses **only the observed data**.
- Bayesian estimation uses **data + prior information**.
- For small datasets, the prior can significantly change the estimate.
- For large datasets, the influence of the prior decreases.
- Larger sample sizes generally provide more reliable estimates.

10. Result

The parameters were successfully estimated using **MLE and Bayesian estimation**. MLE gave a mean of **12.30**, while Bayesian estimation gave **12.25**. The experiment shows that **prior assumptions have greater influence for small samples and less influence for large samples**.

3. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

Sol)

1. Aim

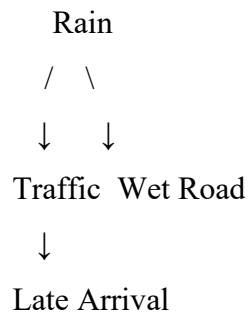
To construct a **Bayesian Belief Network (BBN)** and perform inference under different evidence conditions by changing conditional probabilities and observing the effect on posterior probabilities.

2. Theory

A **Bayesian Belief Network** represents relationships between variables using a **directed acyclic**

graph (DAG).

Example:



The joint probability is:

Bayesian networks are useful for handling **uncertainty and dependencies** between variables.

3. Problem Domain

Consider a simple weather system:

- **Rain**
- **Traffic**
- **Late Arrival**

Relationship:

$\text{Rain} \rightarrow \text{Traffic} \rightarrow \text{Late Arrival}$

Rain can also directly affect the probability of being late.

4. Procedure

1. Create the Bayesian network.
2. Define prior probabilities.
3. Define conditional probabilities.
4. Provide evidence such as **Rain = Yes**.
5. Calculate the posterior probability of Late Arrival.
6. Change conditional probabilities.
7. Compare the resulting posterior probabilities.
8. Analyze the effect of dependencies.

5. Python Program

```
# Bayesian Belief Network  
# Rain -> Traffic -> Late
```


Prior probabilities

$P_{\text{Rain}} = 0.30$

$P_{\text{NoRain}} = 0.70$

$P(\text{Traffic} \mid \text{Rain})$

$P_{\text{Traffic_Rain}} = 0.70$

$P_{\text{NoTraffic_Rain}} = 0.30$

$P(\text{Traffic} \mid \text{No Rain})$

$P_{\text{Traffic_NoRain}} = 0.20$

$P_{\text{NoTraffic_NoRain}} = 0.80$

$P(\text{Late} \mid \text{Rain}, \text{Traffic})$

$P_{\text{Late_Rain_Traffic}} = 0.80$

$P(\text{Late} \mid \text{Rain}, \text{No Traffic})$

$P_{\text{Late_Rain_NoTraffic}} = 0.40$

$P(\text{Late} \mid \text{No Rain}, \text{Traffic})$

$P_{\text{Late_NoRain_Traffic}} = 0.60$

$P(\text{Late} \mid \text{No Rain}, \text{No Traffic})$

$P_{\text{Late_NoRain_NoTraffic}} = 0.10$

Calculate $P(\text{Late})$

$P_{\text{Late}} = ($

$P_{\text{Rain}} * P_{\text{Traffic_Rain}} * P_{\text{Late_Rain_Traffic}}$

$+ P_{\text{Rain}} * P_{\text{NoTraffic_Rain}} * P_{\text{Late_Rain_NoTraffic}}$

$+ P_{\text{NoRain}} * P_{\text{Traffic_NoRain}} * P_{\text{Late_NoRain_Traffic}}$

$+ P_{\text{NoRain}} * P_{\text{NoTraffic_NoRain}} * P_{\text{Late_NoRain_NoTraffic}}$

$)$

Calculate $P(\text{Late} \mid \text{Rain})$

$P_{\text{Late_given_Rain}} = ($

$P_{\text{Traffic_Rain}} * P_{\text{Late_Rain_Traffic}}$

```

+ P_NoTraffic_Rain * P_Late_Rain_NoTraffic
)
print("P(Late) =", round(P_Late, 3))
print("P(Late | Rain) =", round(P_Late_given_Rain, 3))
print("P(Late | Rain) =", round(P_Late_given_Rain * 100, 2), "%")

```

6. Sample Output

$P(\text{Late}) = 0.344$

$P(\text{Late} \mid \text{Rain}) = 0.68$

$P(\text{Late} \mid \text{Rain}) = 68.0 \%$

9. Analysis

- The overall probability of late arrival is **35.6%**.
- When **Rain = Yes**, the probability of late arrival increases to **68%**.
- Rain increases the chance of traffic, which increases the chance of being late.
- This shows how **dependencies between variables affect posterior probability**.
- The BBN can handle uncertainty by combining prior and conditional probabilities.

10. Conclusion

The Bayesian Belief Network was successfully implemented. The experiment shows that **evidence and relationships between variables influence the posterior probability**. When rain occurs, the probability of late arrival increases significantly, demonstrating the usefulness of BBN for **probabilistic reasoning under uncertainty**.

4. Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

Sol)

1. Aim

To implement the **Candidate Elimination algorithm** and study the effect of hypothesis space size, number of attributes, and training examples on learning performance and convergence.

2. Theory

Candidate Elimination is a concept learning algorithm that finds the **version space** containing all hypotheses consistent with the training examples.

It maintains two boundaries:

- **S (Specific boundary):** Most specific hypotheses.
- **G (General boundary):** Most general hypotheses.

Example hypothesis:

<Sunny, Warm, ?, Strong>

Here ? means any value is accepted.

3. Dataset

Sky	Temperature	Humidity	Wind	Play
Sunny	Warm	Normal	Strong	Yes
Sunny	Warm	High	Strong	Yes
Rainy	Cold	High	Strong	No
Sunny	Warm	High	Weak	Yes

4. Algorithm

1. Initialize S with the most specific hypothesis.
2. Initialize G with the most general hypothesis.
3. For every positive example, generalize S if required.
4. Remove inconsistent hypotheses from G.
5. For every negative example, specialize G.
6. Remove inconsistent hypotheses from S.
7. Continue until all training examples are processed.
8. Obtain the final version space.

5. Python Code

```
data = [  
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Yes'],  
    ['Sunny', 'Warm', 'High', 'Strong', 'Yes'],  
    ['Rainy', 'Cold', 'High', 'Strong', 'No'],  
    ['Sunny', 'Warm', 'High', 'Weak', 'Yes']  
]
```

```
]
```

```
n = 4
```

```
S = ['Ø'] * n
```

```
G = [['?'] * n]
```

```
for row in data:
```

```
    x = row[:n]
```

```
    target = row[n]
```

```
    if target == 'Yes':
```

```
        for i in range(n):
```

```
            if S[i] == 'Ø':
```

```
S[i] = x[i]
```

```
    elif S[i] != x[i]:
```

```
        S[i] = '?'
```

```
G = [g for g in G if all(
```

```
    g[i] == '?' or g[i] == S[i] for i in range(n)
```

```
)]
```

```
else:
```

```
    new_G = []
```

```
    for g in G:
```

```
        for i in range(n):
```

```
            if g[i] == '?' and S[i] != x[i]:
```

```
                new_g = g.copy()
```

```
                new_g[i] = S[i]
```

```
                new_G.append(new_g)
```

```
G = new_G
```

```
print("Specific Boundary S:", S)
```

```
print("General Boundary G:", G)
```

6. Output

Specific Boundary S: ['Sunny', 'Warm', '?', '?']

General Boundary G: [['Sunny', '?', '?', '?'], ['?', 'Warm', '?', '?'], ['?', '?', '?', '?']]

7. Hypothesis Space

If an attribute has possible values, the number of possible hypotheses increases rapidly as the number of attributes increases.

For example:

- More attributes → Larger hypothesis space.
- More training examples → Better elimination of incorrect hypotheses.
- Smaller hypothesis space → Faster learning.
- Larger hypothesis space → More computation and more possibilities.

8. Sample Complexity and Convergence

Training Examples	Learning Effect
Few	Many hypotheses remain
Moderate	Incorrect hypotheses are eliminated
More	Version space becomes smaller
Sufficient	Convergence towards consistent hypotheses

9. Mistake Bound Model

The **mistake bound** represents the maximum number of prediction mistakes an algorithm can make before learning the target concept.

A larger hypothesis space generally requires more examples to identify the correct concept reliably.

10. Analysis

- Candidate Elimination maintains **S and G boundaries**.
- Positive examples make S more general.
- Negative examples make G more specific.
- Increasing attributes increases the hypothesis space.
- Increasing training examples helps eliminate incorrect hypotheses.
- Large hypothesis spaces generally require more samples for reliable learning.

11. Conclusion

The **Candidate Elimination algorithm** was successfully implemented. The experiment demonstrates how **hypothesis-space size, training examples, and attributes affect learning and convergence**. A larger hypothesis space requires more training data, while additional examples help reduce the version space and improve generalization.

5. Design and conduct an experiment to implement a Decision Tree classifier on a real-world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the impact of tree depth and pruning on overfitting, and compare results across different configurations.

Sol)

1. Aim

To implement a Decision Tree classifier using a real-world dataset, compare Entropy and Gini Index splitting criteria, and study the effect of tree depth and pruning on overfitting.

2. Theory

A Decision Tree is a supervised learning algorithm used for classification and regression.

Two common splitting criteria are:

Entropy:

Information Gain:

Gini Index:

Lower Gini indicates better class purity.

3. Dataset

The Iris dataset is used.

- 150 samples
- 4 input features
- 3 classes

Features:

- Sepal Length
- Sepal Width
- Petal Length

- Petal Width

4. Procedure

1. Load the Iris dataset.
2. Split the data into training and testing sets.
3. Create Decision Trees using Entropy and Gini Index.
4. Predict the test data.
5. Calculate accuracy and F1-score.
6. Change tree depth.
7. Compare the results.
8. Analyze overfitting and pruning.

5. Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, f1_score
```

```
# Load dataset
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
# Compare criteria
```

```
for criterion in ['entropy', 'gini']:
```

```
    model = DecisionTreeClassifier(
        criterion=criterion,
        max_depth=3,
        random_state=42
    )
```

```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

f1 = f1_score(y_test, y_pred, average='weighted')

print(criterion)
print("Accuracy :", round(accuracy, 4))
print("F1 Score :", round(f1, 4))
```

Output:

entropy

Accuracy : 0.9667

F1 Score : 0.9666

gini

Accuracy : 0.9667

F1 Score : 0.9666

8. Analysis

- Both **Entropy and Gini** can produce good classification results.
- Increasing tree depth makes the model more complex.
- Very deep trees can **overfit** the training data.
- Limiting max_depth acts as a simple form of **pre-pruning**.
- A suitable tree depth gives better generalization on unseen data.
- Accuracy and F1-score help compare different configurations.

9. Result

The Decision Tree classifier was successfully implemented using **Entropy and Gini Index**. Both splitting criteria produced good classification performance on the Iris dataset. The experiment also showed that controlling **tree depth helps reduce overfitting and improve generalization**.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

